T.Y.B.Sc. Cyber & Digital Science — Web Science (CDS-366)

Practical Slip Solutions — Sem VI (April 2024-25)

Savitribai Phule Pune University | MIT Arts, Commerce and Science College

---

Slip Index & Repeat Map

Questions repeated across slips are solved ONCE and referenced elsewhere.

Slip	Q1	Q2	Status
1	Student Profile Webpage (2-col layout)	Login Form with JS Validation	Solved
2	Create Subject.xml with 3 records	SQL Injection on Login Form	Solved
3	cricket.xml + add India team	Dairy Product Web Service (REST)	Solved
4	Online Weather API Access	Password Strength Validator	Solved
5	Registration with encrypted password	Student Profile (REPEAT Slip 1 Q1)	Partial
6	PHP Palindrome check	Car XML with XSLT display	Solved
7	DB Relations + SQL Queries	Weather API (REPEAT Slip 4 Q1)	Partial
8	HTML 7 colored lines	Password Validator (REPEAT Slip 4 Q2)	Partial
9	Bus Ticket Reservation Form	Customer Registration Form	Solved
10	Student.xml (5 records)	Employees.xml (5 records)	Solved
11	PHP Session Login (3 chances)	PHP Multi-page Employee Form	Solved
12	Cylinder Area/Volume (PHP)	String Operations (PHP)	Solved
13	Number to Words (PHP)	REPEAT of Q1	Partial
14	Product List Cart (PHP)	OWASP Top 10 Web Threats	Solved
15	Security Threat Report	Timetable.xml	Solved
16	SQL Injection Login (REPEAT Slip 2 Q2)	Burp Suite Scanner Note	Partial
17	Product Cart (REPEAT Slip 14 Q1)	Course Registrations.xml	Partial
18	breakfast.xml + add Juice	SQL Injection UPDATE query	Solved
19	Book Form + MySQL Store (PHP)	Dairy Product Service (REPEAT Slip 3 Q2)	Partial
20	Library Book Web Service	Secure User Registration	Solved
21	Factorial using PHP function	OWASP ZAP Scanner Note	Solved
22	OWASP Top 10 API Threats	Password Validator (REPEAT Slip 4 Q2)	Partial
23	Employee Profile Webpage	Feedback Form with Star Rating	Solved
24	Timetable.xml (REPEAT Slip 15 Q2)	Currency Exchange API	Partial
25	SQL Injection UPDATE (REPEAT Slip 18 Q2)	cricket.xml Australia (REPEAT Slip 3)	Repeat
26	Student.xml (REPEAT Slip 10 Q1)	Library Book Service (REPEAT Slip 20 Q1)	Repeat

---

SLIP 1

Q1 — Student Profile Webpage (2-Column Layout)

File: student_profile.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student Profile</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 0; }
    .container { display: flex; height: 100vh; }
    /* Column 1 - Navigation */
    .nav { width: 200px; background: #1a3c5e; padding: 20px; }
    .nav a { display: block; color: white; text-decoration: none;
             padding: 10px; margin: 5px 0; border-radius: 4px; }
    .nav a:hover { background: #2e75b6; }
    /* Column 2 - Profile */
    .profile { flex: 1; padding: 30px; background: #f0f4f8; }
    .profile img { width: 120px; height: 120px; border-radius: 50%;
                   object-fit: cover; border: 3px solid #2e75b6; }
    table { border-collapse: collapse; margin-top: 15px; }
    td { padding: 8px 15px; border-bottom: 1px solid #ccc; }
    td:first-child { font-weight: bold; color: #1a3c5e; }
  </style>
</head>
<body>
  <div class="container">
    <!-- Column 1: Navigation Links -->
    <div class="nav">
      <h3 style="color:white;">Menu</h3>
      <a href="#">□ Home</a>
      <a href="#">□ Contact Us</a>
      <a href="#">□ About Us</a>
    </div>

    <!-- Column 2: Student Profile -->
    <div class="profile">
      <h2>Student Profile</h2>
      <img src="student.jpg" alt="Student Photo">
      <table>
        <tr><td>Roll Number</td><td>2201</td></tr>
        <tr><td>Name</td><td>Rushikesh Bhosale</td></tr>
        <tr><td>Contact</td><td>9876543210</td></tr>
        <tr><td>Class</td><td>T.Y.B.Sc. (Cyber & Digital Science)</td></tr>
        <tr><td>Area of Interest</td><td>Cybersecurity, SOC Analysis</td></tr>
      </table>
    </div>
  </div>
</body>
</html>
```

□ *Replace "student.jpg" with your actual image file. The layout uses CSS Flexbox for 2-column design.*

Q2 — Login Form with JS Validation

File: login.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Login Form</title>
  <style>
    body { font-family: Arial, sans-serif; display:flex;
           justify-content:center; align-items:center; height:100vh;
           background:#f0f4f8; }
    .box { background:white; padding:30px; border-radius:8px;
           box-shadow:0 2px 10px rgba(0,0,0,0.1); width:300px; }
    input { width:100%; padding:8px; margin:8px 0; box-sizing:border-box;
            border:1px solid #ccc; border-radius:4px; }
    button { width:100%; padding:10px; background:#2e75b6;
             color:white; border:none; border-radius:4px; cursor:pointer; }
    #message { margin-top:10px; font-weight:bold; text-align:center; }
  </style>
</head>
<body>
  <div class="box">
    <h2>Login</h2>
    <input type="text"     id="username" placeholder="Username">
    <input type="password" id="password" placeholder="Password">
    <button onclick="checkLogin()">Login</button>
    <p id="message"></p>
  </div>

  <script>
    function checkLogin() {
      const user = document.getElementById('username').value;
      const pass = document.getElementById('password').value;
      const msg  = document.getElementById('message');

      if (user === pass && user !== '') {
        msg.style.color = 'green';
        msg.textContent = 'Welcome!';
      } else {
        msg.style.color = 'red';
        msg.textContent = 'Invalid username or password';
      }
    }
  </script>
</body>
</html>
```

Logic: If username === password (and not empty), show "Welcome" — else show "Invalid username or password".

SLIP 2

Q1 — Create Subject.xml with at least 3 records

File: Subject.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Subjects>
  <Subject>
    <SubjectID>CDS301</SubjectID>
    <SubjectName>Web Science</SubjectName>
    <Class>T.Y.B.Sc.</Class>
    <Semester>VI</Semester>
  </Subject>
  <Subject>
    <SubjectID>CDS302</SubjectID>
    <SubjectName>Cyber Security</SubjectName>
    <Class>T.Y.B.Sc.</Class>
    <Semester>VI</Semester>
  </Subject>
  <Subject>
    <SubjectID>CDS303</SubjectID>
    <SubjectName>Digital Forensics</SubjectName>
    <Class>T.Y.B.Sc.</Class>
    <Semester>VI</Semester>
  </Subject>
</Subjects>
```

Q2 — SQL Injection on Login Form

Concept: SQL Injection bypasses authentication by injecting SQL commands.

Vulnerable PHP Login Code (to understand the attack):

```
// Vulnerable login query:
$query = "SELECT * FROM users WHERE username='$user' AND password='$pass'";

// Normal input:
//   username = admin
//   password = secret123
//   → SELECT * FROM users WHERE username='admin' AND password='secret123'

// SQL Injection input:
//   username = admin'--
//   password = (anything)
//   → SELECT * FROM users WHERE username='admin'--' AND password='...'
//   The '--' comments out the password check → login succeeds without password!
```

Common SQL Injection payloads for login:

```
Username: ' OR '1'='1
Password: ' OR '1'='1
→ Query becomes: WHERE username='' OR '1'='1' AND password='' OR '1'='1'
→ Always TRUE → logged in as first user in DB

Username: admin'--
Password: anything
→ Query becomes: WHERE username='admin'--' AND password='...'
→ Password check skipped → logged in as admin
```

Prevention (Parameterized Query in PHP):

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username=? AND password=?");
$stmt->execute([$username, $password]);
```

Always use prepared statements / parameterized queries to prevent SQL injection.

SLIP 3

Q1 — cricket.xml with Australia team + add India records

```
<?xml version="1.0" encoding="UTF-8"?>
<CricketTeams>
  <Team Country="Australia">
    <Player>David Warner</Player>
    <Runs>8786</Runs>
    <Wicket>1</Wicket>
  </Team>
  <Team Country="Australia">
    <Player>Pat Cummins</Player>
    <Runs>1200</Runs>
    <Wicket>250</Wicket>
  </Team>
  <Team Country="India">
    <Player>Virat Kohli</Player>
    <Runs>12000</Runs>
    <Wicket>0</Wicket>
  </Team>
  <Team Country="India">
    <Player>Jasprit Bumrah</Player>
    <Runs>300</Runs>
    <Wicket>180</Wicket>
  </Team>
</CricketTeams>
```

First two <Team> blocks = Australia (original). Last two = India records added via script.

Q2 — Dairy Product Price Web Service (REST API using PHP)

File: dairy_service.php

```
<?php
// Simple REST-like service for Dairy Product Prices
// Using a JSON file as storage (no DB needed for demo)

$file = 'dairy_products.json';

// Load existing products
$products = file_exists($file) ? json_decode(file_get_contents($file), true) : [];

$action = $_GET['action'] ?? '';
$id     = $_GET['id']     ?? null;
$name   = $_POST['name']  ?? '';
$price  = $_POST['price'] ?? '';

switch ($action) {
  case 'add':
    $products[] = ['id' => time(), 'name' => $name, 'price' => $price];
    file_put_contents($file, json_encode($products));
    echo json_encode(['status' => 'added']);
    break;

  case 'delete':
    $products = array_filter($products, fn($p) => $p['id'] != $id);
    file_put_contents($file, json_encode(array_values($products)));
    echo json_encode(['status' => 'deleted']);
```

```
    break;

  case 'update':
    foreach ($products as &$p) {
      if ($p['id'] == $id) { $p['name'] = $name; $p['price'] = $price; break; }
    }
    file_put_contents($file, json_encode($products));
    echo json_encode(['status' => 'updated']);
    break;

  default:
    echo json_encode($products);  // List all products
}
?>
```

Usage examples (via URL / form POST):

```
List all:   dairy_service.php
Add:        dairy_service.php?action=add   POST: name=Milk&price=50
Delete:     dairy_service.php?action=delete&id=1234567890
Update:     dairy_service.php?action=update&id=1234567890 POST: name=Butter&price=80
```

SLIP 4

Q1 — Access Online Weather API

File: weather.php

```php
<?php
// Using OpenWeatherMap Free API (sign up at openweathermap.org for free key)
$apiKey = "YOUR_API_KEY_HERE";
$city   = isset($_GET['city']) ? $_GET['city'] : "Pune";

$url  =
"https://api.openweathermap.org/data/2.5/weather?q={$city}&appid={$apiKey}&units=metric";
$res  = file_get_contents($url);
$data = json_decode($res, true);

if ($data['cod'] == 200) {
  echo "<h2>Weather in {$data['name']}, {$data['sys']['country']}</h2>";
  echo "<p>Temperature : {$data['main']['temp']} &deg;C</p>";
  echo "<p>Feels Like  : {$data['main']['feels_like']} &deg;C</p>";
  echo "<p>Humidity    : {$data['main']['humidity']}%</p>";
  echo "<p>Condition   : {$data['weather'][0]['description']}</p>";
  echo "<p>Wind Speed  : {$data['wind']['speed']} m/s</p>";
} else {
  echo "<p>City not found. Try again.</p>";
}
?>

<!-- HTML form to accept city input -->
<form method="GET">
  <input type="text" name="city" placeholder="Enter city name">
  <button type="submit">Get Weather</button>
</form>
```

Get a free API key from openweathermap.org. Replace YOUR_API_KEY_HERE.

Q2 — Password Strength Validator

File: password_check.html

```html
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Password Checker</title></head>
<body>
  <h2>Password Strength Checker</h2>
  <input type="text" id="uname" placeholder="Username"><br><br>
  <input type="password" id="pass" placeholder="Password"><br><br>
  <button onclick="checkPass()">Check</button>
  <p id="result"></p>

  <script>
    function checkPass() {
      const pass = document.getElementById('pass').value;
      const result = document.getElementById('result');

      const hasNum     = /[0-9]/.test(pass);
      const hasAlpha   = /[a-zA-Z]/.test(pass);
      const hasSpecial = /[!@#$%^&*(),.?":{}|<>]/.test(pass);
```

```
      const hasLength  = pass.length > 8;

      if (hasNum && hasAlpha && hasSpecial && hasLength) {
        result.style.color = 'green';
        result.textContent = 'You entered a strong password';
      } else {
        result.style.color = 'red';
        result.textContent = 'Enter complex password (8+ chars, letters, numbers &
special chars)';
      }
    }
  </script>
</body>
</html>
```

REPEATED: Slip 8 Q2, Slip 22 Q2 — same question. Refer to this solution.

SLIP 5

Q1 — Registration with Encrypted Password Storage

File: register.php + register.html

register.html (form):

```
<!DOCTYPE html>
<html><head><meta charset="UTF-8"><title>Register</title></head>
<body>
<h2>User Registration</h2>
<form action="register.php" method="POST">
  Full Name   : <input type="text"     name="fname"><br><br>
  Contact No  : <input type="text"     name="contact"><br><br>
  Email ID    : <input type="email"    name="email"><br><br>
  Password    : <input type="password" name="pass"><br><br>
  Confirm Pass: <input type="password" name="cpass"><br><br>
  <button type="submit">Register</button>
</form>
</body></html>
```

register.php (backend):

```
<?php
$fname   = $_POST['fname'];
$contact = $_POST['contact'];
$email   = $_POST['email'];
$pass    = $_POST['pass'];
$cpass   = $_POST['cpass'];

if ($pass !== $cpass) {
  die("Error: Passwords do not match!");
}

// Encrypt password using bcrypt
$hashed = password_hash($pass, PASSWORD_BCRYPT);

// Connect to MySQL
$conn = new mysqli("localhost", "root", "", "userdb");
if ($conn->connect_error) { die("Connection failed: " . $conn->connect_error); }

// Store in database
$stmt = $conn->prepare("INSERT INTO users (fname, contact, email, password) VALUES
(?,?,?,?)");
$stmt->bind_param("ssss", $fname, $contact, $email, $hashed);

if ($stmt->execute()) {
  echo "Registration successful! Password stored securely.";
} else {
  echo "Error: " . $stmt->error;
}

$stmt->close();
$conn->close();
?>
```

REPEAT: Q2 of this slip is the same as Slip 1 Q1 (Student Profile). Refer to Slip 1 Q1.

SLIP 6

Q1 — PHP Palindrome Check

File: palindrome.html + palindrome.php

palindrome.html:

```
<form action="palindrome.php" method="POST">
  Enter a number: <input type="number" name="num">
  <button type="submit">Check</button>
</form>
```

palindrome.php:

```
<?php
$num = $_POST['num'];
$rev = strrev($num);           // Reverse the number string

if ($num == $rev) {
  echo "$num is a PALINDROME";
} else {
  echo "$num is NOT a palindrome";
}
?>
```

Q2 — Car XML with XSLT Table Display

cars.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="cars.xsl"?>
<Cars>
  <Car id="1">
    <Company>Toyota</Company>
    <Model>Camry</Model>
    <Engine>2.5L Petrol</Engine>
    <Mileage>16 kmpl</Mileage>
  </Car>
  <Car id="2">
    <Company>Hyundai</Company>
    <Model>Creta</Model>
    <Engine>1.5L Diesel</Engine>
    <Mileage>21 kmpl</Mileage>
  </Car>
  <Car id="3">
    <Company>Maruti</Company>
    <Model>Swift</Model>
    <Engine>1.2L Petrol</Engine>
    <Mileage>23 kmpl</Mileage>
  </Car>
</Cars>
```

cars.xsl (XSLT to display as HTML table):

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
  <html>
  <body>
    <h2>Car Details</h2>
    <table border="1">
```

```
      <tr>
        <th>ID</th><th>Company</th><th>Model</th>
        <th>Engine</th><th>Mileage</th>
      </tr>
      <xsl:for-each select="Cars/Car">
        <tr>
          <td><xsl:value-of select="@id"/></td>
          <td><xsl:value-of select="Company"/></td>
          <td><xsl:value-of select="Model"/></td>
          <td><xsl:value-of select="Engine"/></td>
          <td><xsl:value-of select="Mileage"/></td>
        </tr>
      </xsl:for-each>
    </table>
  </body>
  </html>
</xsl:template>
</xsl:stylesheet>
```

Open cars.xml in a browser — the XSLT automatically transforms and renders the table.

SLIP 7

Q1 — Database Relations + SQL Queries

Tables: Emp(eno, name, dno, salary) | Project(pno, pname, controldno, budget) | Works_On(eno, pno, hours)

CREATE TABLE statements:

```
CREATE TABLE Emp (
  eno       INT PRIMARY KEY,
  name      VARCHAR(50),
  dno       INT,
  salary    DECIMAL(10,2)
);

CREATE TABLE Project (
  pno        INT PRIMARY KEY,
  pname      VARCHAR(50),
  controldno INT,
  budget     DECIMAL(12,2)
);

-- Junction table for many-to-many (Employee works on Project)
CREATE TABLE Works_On (
  eno   INT,
  pno   INT,
  hours INT,
  PRIMARY KEY (eno, pno),
  FOREIGN KEY (eno) REFERENCES Emp(eno),
  FOREIGN KEY (pno) REFERENCES Project(pno)
);
```

Query 1 — Projects with budget > 10000:

```
SELECT pname FROM Project WHERE budget > 10000;
```

Query 2 — Projects controlled by department 101:

```
SELECT pname FROM Project WHERE controldno = 101;
```

Query 3 — Project with second maximum budget:

```
SELECT * FROM Project
ORDER BY budget DESC
LIMIT 1 OFFSET 1;
```

Query 4 — Project with maximum budget:

```
SELECT * FROM Project
WHERE budget = (SELECT MAX(budget) FROM Project);
```

Query 5 — Employees working in E&TC department (dno = E&TC's dno, e.g., 102):

```
SELECT name FROM Emp WHERE dno = 102;
```

REPEAT: Q2 of this slip is 'Weather API' — same as Slip 4 Q1. Refer to Slip 4 Q1.

SLIP 8

Q1 — HTML Page with 7 Colored Lines

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Colored Lines</title></head>
<body>
  <hr style="border: 3px solid red;">
  <p>This line is <strong>Red</strong></p>

  <hr style="border: 3px solid blue;">
  <p>This line is <strong>Blue</strong></p>

  <hr style="border: 3px solid green;">
  <p>This line is <strong>Green</strong></p>

  <hr style="border: 3px solid orange;">
  <p>This line is <strong>Orange</strong></p>

  <hr style="border: 3px solid purple;">
  <p>This line is <strong>Purple</strong></p>

  <hr style="border: 3px solid brown;">
  <p>This line is <strong>Brown</strong></p>

  <hr style="border: 3px solid teal;">
  <p>This line is <strong>Teal</strong></p>
</body>
</html>
```

REPEAT: Q2 of this slip is Password Validator — same as Slip 4 Q2. Refer to Slip 4 Q2.

SLIP 9

Q1 — Bus Ticket Reservation Form

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Bus Ticket Reservation</title>
<style>
  body { font-family: Arial; padding: 20px; background: #f5f5f5; }
  .form-box { background: white; padding: 25px; border-radius: 8px;
              max-width: 600px; margin: auto; box-shadow: 0 2px 8px rgba(0,0,0,0.1); }
  label { display: block; margin-top: 10px; font-weight: bold; }
  input, select { width: 100%; padding: 7px; margin-top: 4px;
                  border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box; }
  button { margin-top: 15px; padding: 10px 25px; background: #2e75b6;
           color: white; border: none; border-radius: 4px; cursor: pointer; }
</style>
</head>
<body>
<div class="form-box">
  <h2>Bus Ticket Reservation</h2>
  <label>Passenger Name</label>
  <input type="text" name="name" placeholder="Full Name">

  <label>Address</label>
  <input type="text" name="address" placeholder="Your Address">

  <label>Contact Number</label>
  <input type="tel" name="contact" placeholder="Mobile Number">

  <label>Source Station</label>
  <select name="source">
    <option>Pune</option>
    <option>Mumbai</option>
    <option>Nashik</option>
    <option>Aurangabad</option>
  </select>

  <label>Destination Station</label>
  <input type="text" name="destination" placeholder="Enter destination">

  <label>Date of Booking</label>
  <input type="date" name="booking_date">

  <label>Date of Journey</label>
  <input type="date" name="journey_date">

  <label>Number of Passengers</label>
  <input type="number" name="num_passengers" min="1" value="1">

  <label>Passenger Name(s)</label>
  <input type="text" name="pname" placeholder="e.g. Rahul, Priya">

  <label>Gender</label>
  <select name="gender">
    <option>Male</option>
    <option>Female</option>
    <option>Other</option>
```

```
  </select>

  <button type="submit">Book Ticket</button>
  <button type="reset" style="background:#888;">Reset</button>
</div>
</body></html>
```

Q2 — Customer Registration Form (Departmental Store)

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Customer Registration</title></head>
<body>
<h2>Customer Registration</h2>
<form>
  Name: <input type="text" name="name"><br><br>

  Contact No: <input type="tel" name="contact"><br><br>

  Gender:
  <input type="radio" name="gender" value="Male"> Male
  <input type="radio" name="gender" value="Female"> Female
  <input type="radio" name="gender" value="Other"> Other<br><br>

  Preferred Shopping Days:
  <input type="checkbox" name="days" value="Mon"> Mon
  <input type="checkbox" name="days" value="Wed"> Wed
  <input type="checkbox" name="days" value="Fri"> Fri
  <input type="checkbox" name="days" value="Sun"> Sun<br><br>

  Favorite Items:
  <select name="fav_item">
    <option>Electronics</option>
    <option>Clothing</option>
    <option>Groceries</option>
    <option>Footwear</option>
    <option>Cosmetics</option>
  </select><br><br>

  Suggestions:<br>
  <textarea name="suggestions" rows="3" cols="40"></textarea><br><br>

  <button type="submit">Submit</button>
  <button type="reset">Reset</button>
</form>
</body></html>
```

SLIP 10

Q1 — Student.xml with 5 records

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Students>
  <Student>
    <StudentID>S001</StudentID>
    <StudentName>Rushikesh Bhosale</StudentName>
    <Age>21</Age>
    <Gender>Male</Gender>
    <Program>B.Sc. Cyber and Digital Science</Program>
  </Student>
  <Student>
    <StudentID>S002</StudentID>
    <StudentName>Priya Sharma</StudentName>
    <Age>20</Age>
    <Gender>Female</Gender>
    <Program>B.Sc. Computer Science</Program>
  </Student>
  <Student>
    <StudentID>S003</StudentID>
    <StudentName>Rohit Patil</StudentName>
    <Age>22</Age>
    <Gender>Male</Gender>
    <Program>B.Sc. Cyber and Digital Science</Program>
  </Student>
  <Student>
    <StudentID>S004</StudentID>
    <StudentName>Sneha Kulkarni</StudentName>
    <Age>21</Age>
    <Gender>Female</Gender>
    <Program>B.Sc. Information Technology</Program>
  </Student>
  <Student>
    <StudentID>S005</StudentID>
    <StudentName>Aman Desai</StudentName>
    <Age>23</Age>
    <Gender>Male</Gender>
    <Program>B.Sc. Cyber and Digital Science</Program>
  </Student>
</Students>
```

REPEAT: Slip 26 Q1 is exactly this. Refer here.

Q2 — Employees.xml with 5 records

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Employees>
  <Employee>
    <EmployeeID>E001</EmployeeID>
    <Name>Anjali Mehta</Name>
    <Position>SOC Analyst</Position>
    <Department>Cybersecurity</Department>
    <DateOfJoining>2022-06-15</DateOfJoining>
  </Employee>
  <Employee>
```

```xml
    <EmployeeID>E002</EmployeeID>
    <Name>Kiran Joshi</Name>
    <Position>Network Engineer</Position>
    <Department>IT Infrastructure</Department>
    <DateOfJoining>2021-03-10</DateOfJoining>
  </Employee>
  <Employee>
    <EmployeeID>E003</EmployeeID>
    <Name>Neha Thakur</Name>
    <Position>Web Developer</Position>
    <Department>Software Development</Department>
    <DateOfJoining>2023-01-20</DateOfJoining>
  </Employee>
  <Employee>
    <EmployeeID>E004</EmployeeID>
    <Name>Raj Verma</Name>
    <Position>Penetration Tester</Position>
    <Department>Cybersecurity</Department>
    <DateOfJoining>2020-09-05</DateOfJoining>
  </Employee>
  <Employee>
    <EmployeeID>E005</EmployeeID>
    <Name>Pooja Nair</Name>
    <Position>HR Manager</Position>
    <Department>Human Resources</Department>
    <DateOfJoining>2019-11-30</DateOfJoining>
  </Employee>
</Employees>
```

SLIP 11

Q1 — PHP Session Login (3 Chances)

File: login.php

```php
<?php
session_start();

// Initialize attempt counter
if (!isset($_SESSION['attempts'])) {
  $_SESSION['attempts'] = 0;
}

$correct_user = "admin";
$correct_pass = "admin123";
$message = "";

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  if ($_SESSION['attempts'] >= 3) {
    $message = "Account locked! Max 3 attempts reached.";
  } else {
    $user = $_POST['username'];
    $pass = $_POST['password'];

    if ($user === $correct_user && $pass === $correct_pass) {
      $_SESSION['logged_in'] = true;
      $_SESSION['attempts']  = 0;
      header("Location: welcome.php");
      exit();
    } else {
      $_SESSION['attempts']++;
      $remaining = 3 - $_SESSION['attempts'];
      $message = "Invalid credentials! Attempts left: $remaining";
    }
  }
}
?>
<!DOCTYPE html>
<html><head><title>Login</title></head>
<body>
  <h2>Login (Max 3 Attempts)</h2>
  <p style="color:red;"><?= $message ?></p>
  <form method="POST">
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
    <button type="submit">Login</button>
  </form>
</body></html>
```

welcome.php:

```php
<?php
session_start();
if (!isset($_SESSION['logged_in'])) {
  header("Location: login.php"); exit();
}
?>
<h2>Welcome! You are logged in successfully.</h2>
<a href="logout.php">Logout</a>
```

Q2 — PHP Multi-page Employee Form (Sessions)

page1.php — Accept Eno, Ename, Address:

```
<?php session_start(); ?>
<form action="page2.php" method="POST">
  Emp No  : <input type="text" name="eno"><br>
  Emp Name: <input type="text" name="ename"><br>
  Address : <input type="text" name="address"><br>
  <button type="submit">Next</button>
</form>
```

page2.php — Accept Basic, DA, HRA:

```
<?php
session_start();
$_SESSION['eno']     = $_POST['eno'];
$_SESSION['ename']   = $_POST['ename'];
$_SESSION['address'] = $_POST['address'];
?>
<form action="page3.php" method="POST">
  Basic: <input type="number" name="basic"><br>
  DA   : <input type="number" name="da"><br>
  HRA  : <input type="number" name="hra"><br>
  <button type="submit">View Summary</button>
</form>
```

page3.php — Print full employee summary:

```
<?php
session_start();
$basic = $_POST['basic'];
$da    = $_POST['da'];
$hra   = $_POST['hra'];
$total = $basic + $da + $hra;
?>
<h2>Employee Information</h2>
<table border="1" cellpadding="8">
  <tr><td>Emp No</td>  <td><?= $_SESSION['eno'] ?></td></tr>
  <tr><td>Name</td>    <td><?= $_SESSION['ename'] ?></td></tr>
  <tr><td>Address</td> <td><?= $_SESSION['address'] ?></td></tr>
  <tr><td>Basic</td>   <td><?= $basic ?></td></tr>
  <tr><td>DA</td>      <td><?= $da ?></td></tr>
  <tr><td>HRA</td>     <td><?= $hra ?></td></tr>
  <tr><td><strong>Total</strong></td><td><strong><?= $total ?></strong></td></tr>
</table>
```

SLIP 12

Q1 — Cylinder Area & Volume Calculator (PHP)

cylinder.html:

```
<form action="cylinder.php" method="POST">
  Radius : <input type="number" step="0.01" name="r"><br>
  Height : <input type="number" step="0.01" name="h"><br>
  <button type="submit">Calculate</button>
</form>
```

cylinder.php:

```
<?php
function cylinder_area($r, $h) {
  return 2 * M_PI * $r * ($r + $h);   // Total Surface Area
}
function cylinder_volume($r, $h) {
  return M_PI * $r * $r * $h;
}

$r = $_POST['r'];
$h = $_POST['h'];

$area   = round(cylinder_area($r, $h), 2);
$volume = round(cylinder_volume($r, $h), 2);

echo "Radius : $r | Height : $h<br>";
echo "Total Surface Area : $area sq. units<br>";
echo "Volume             : $volume cubic units";
?>
```

Q2 — String Operations (PHP + Radio Button)

strings.html:

```
<form action="strings.php" method="POST">
  String 1: <input type="text" name="s1"><br>
  String 2: <input type="text" name="s2"><br>
  <input type="radio" name="op" value="compare">  Compare Strings<br>
  <input type="radio" name="op" value="upper">    Convert to Uppercase<br>
  <input type="radio" name="op" value="lower">    Convert to Lowercase<br>
  <button type="submit">Perform</button>
</form>
```

strings.php:

```
<?php
$s1 = $_POST['s1'];
$s2 = $_POST['s2'];
$op = $_POST['op'];

switch ($op) {
  case 'compare':
    $result = (strcmp($s1, $s2) === 0) ? "Strings are EQUAL" : "Strings are NOT equal";
    break;
  case 'upper':
    $result = "Uppercase: " . strtoupper($s1) . " | " . strtoupper($s2);
    break;
  case 'lower':
    $result = "Lowercase: " . strtolower($s1) . " | " . strtolower($s2);
```

```
    break;
  default:
    $result = "Please select an operation.";
}
echo $result;
?>
```

SLIP 13

Q1 & Q2 — Number to Words (Both questions are the same)

Note: Q1 and Q2 in Slip 13 are identical. Solved once below.

numwords.html:

```
<form action="numwords.php" method="POST">
  Enter Number: <input type="number" name="num">
  <button type="submit">Show in Words</button>
</form>
```

numwords.php:

```
<?php
$words = ["Zero","One","Two","Three","Four",
          "Five","Six","Seven","Eight","Nine"];

$num = strval($_POST['num']);   // Convert to string to iterate digits

echo "Number: $num<br>Digits in words: ";
for ($i = 0; $i < strlen($num); $i++) {
  echo $words[(int)$num[$i]] . " ";
}
?>

<!-- Example: input 4752 → Output: Four Seven Five Two -->
```

SLIP 14

Q1 — PHP Product Selection Cart

File: cart.php (single file)

```
<?php
$products = [
  'Book' => 150,
  'Pen'  => 20,
  'Bag'  => 500
];

$total_qty  = 0;
$total_cost = 0;
$selected   = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  foreach ($products as $item => $price) {
    $qty_key = strtolower($item) . '_qty';
    $qty = isset($_POST[$qty_key]) ? (int)$_POST[$qty_key] : 0;
    if ($qty > 0) {
      $selected[]  = ['name' => $item, 'price' => $price, 'qty' => $qty, 'cost' => $price
* $qty];
      $total_qty  += $qty;
      $total_cost += $price * $qty;
    }
  }
}
?>
<!DOCTYPE html>
<html><head><title>Product Cart</title></head>
<body>
<h2>Select Products</h2>
<form method="POST">
  <table border="1" cellpadding="8">
    <tr><th>Item</th><th>Price</th><th>Quantity</th></tr>
    <?php foreach ($products as $item => $price): ?>
    <tr>
      <td><?= $item ?></td>
      <td>Rs. <?= $price ?></td>
      <td><input type="number" name="<?= strtolower($item) ?>_qty" value="0"
min="0"></td>
    </tr>
    <?php endforeach; ?>
  </table>
  <button type="submit">Calculate Total</button>
</form>

<?php if (!empty($selected)): ?>
<h3>Order Summary</h3>
<table border="1" cellpadding="8">
  <tr><th>Item</th><th>Unit Price</th><th>Qty</th><th>Cost</th></tr>
  <?php foreach ($selected as $row): ?>
  <tr>
    <td><?= $row['name'] ?></td>
    <td>Rs. <?= $row['price'] ?></td>
    <td><?= $row['qty'] ?></td>
    <td>Rs. <?= $row['cost'] ?></td>
  </tr>
```

```
  <?php endforeach; ?>
  <tr><td colspan="2"><strong>Total</strong></td>
      <td><strong><?= $total_qty ?></strong></td>
      <td><strong>Rs. <?= $total_cost ?></strong></td></tr>
</table>
<?php endif; ?>
</body></html>
```

REPEAT: Slip 17 Q1 is identical to this. Refer here.

Q2 — OWASP Top 10 Web Application Threats

The OWASP Top 10 (2021) for Web Applications:

```
1. A01 - Broken Access Control
   Users access resources/functions beyond their permitted level.
   Example: A normal user accessing admin panels.

2. A02 - Cryptographic Failures
   Sensitive data exposed due to weak/missing encryption.
   Example: Passwords stored as plain text in database.

3. A03 - Injection
   Attacker injects malicious code (SQL, OS, LDAP commands).
   Example: SQL injection via login form.

4. A04 - Insecure Design
   Design-level flaws that allow security gaps.
   Example: No rate limiting on OTP verification.

5. A05 - Security Misconfiguration
   Default credentials, open cloud storage, verbose error messages.
   Example: Admin panel accessible with default admin/admin.

6. A06 - Vulnerable and Outdated Components
   Using libraries/frameworks with known vulnerabilities.
   Example: Running an outdated WordPress with unpatched plugins.

7. A07 - Identification and Authentication Failures
   Weak session management, missing MFA.
   Example: Session tokens that don't expire.

8. A08 - Software and Data Integrity Failures
   Untrusted data in CI/CD pipelines or auto-updates.
   Example: Malicious package injected via NPM dependency.

9. A09 - Security Logging and Monitoring Failures
   Attacks not detected due to lack of logs/alerts.
   Example: Failed login attempts not logged or alerted.

10. A10 - Server-Side Request Forgery (SSRF)
    Server fetches attacker-controlled URLs.
    Example: Attacker causes server to call internal AWS metadata endpoint.
```

SLIP 15

Q1 — Security Threat Report (Short Report Template)

Report Format:

```
SECURITY THREAT REPORT
======================

a) Identified Threat:
   SQL Injection vulnerability found in the login form of the web application.
   The username field was not sanitized, allowing raw SQL to be injected.

b) How the Threat Was Exploited:
   Entered the payload:  admin'--  in the username field with any password.
   The injected query became:
     SELECT * FROM users WHERE username='admin'--' AND password='...'
   The double dash (--) commented out the password check.
   Result: Logged in as 'admin' without knowing the password.

c) Mitigation Steps Taken:
   1. Replaced string-concatenated queries with parameterized queries (PDO/MySQLi).
   2. Applied input validation and sanitization using htmlspecialchars() and trim().
   3. Enabled error suppression in production (display_errors = Off).
   4. Implemented Web Application Firewall (WAF) rules.
   5. Conducted code review and pen-test using OWASP ZAP.
```

Q2 — Timetable.xml (3 records)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Timetable>
  <Class>
    <Day>Monday</Day>
    <ClassName>Web Science Lab</ClassName>
    <Instructor>Prof. Sharma</Instructor>
    <TimeSlot>10:00 AM - 01:00 PM</TimeSlot>
    <Classroom>Lab 3</Classroom>
  </Class>
  <Class>
    <Day>Wednesday</Day>
    <ClassName>Cyber Security</ClassName>
    <Instructor>Prof. Kulkarni</Instructor>
    <TimeSlot>11:00 AM - 12:00 PM</TimeSlot>
    <Classroom>Room 205</Classroom>
  </Class>
  <Class>
    <Day>Friday</Day>
    <ClassName>Digital Forensics</ClassName>
    <Instructor>Prof. Desai</Instructor>
    <TimeSlot>02:00 PM - 03:00 PM</TimeSlot>
    <Classroom>Room 301</Classroom>
  </Class>
</Timetable>
```

REPEAT: Slip 24 Q1 is identical to this. Refer here.

SLIP 16

Q1 — SQL Injection on Login

REPEAT: Same as Slip 2 Q2. Refer to Slip 2 Q2 for full solution and payloads.

Q2 — Burp Suite Scanner: Short Note with Example

```
BURP SUITE SCANNER — SHORT NOTE
================================

Definition:
  Burp Suite is a leading web application security testing platform
  developed by PortSwigger. It acts as a proxy between the browser
  and the web server, allowing testers to intercept, modify, and
  replay HTTP/HTTPS requests.

Key Features:
  - Proxy     : Intercept and modify browser-server traffic
  - Scanner   : Automatically detect vulnerabilities (SQLi, XSS, etc.)
  - Repeater  : Manually replay modified requests
  - Intruder  : Automate attacks (brute-force, fuzzing)
  - Decoder   : Encode/decode data (Base64, URL, etc.)

Example — Detecting SQL Injection:
  Step 1: Configure browser proxy to 127.0.0.1:8080
  Step 2: Open Burp Suite → Proxy → Intercept ON
  Step 3: Submit login form in browser
  Step 4: Burp captures the POST request
  Step 5: Send to Scanner / Repeater
  Step 6: Modify username parameter → add payload: admin'--
  Step 7: Forward request and observe response
  Step 8: If logged in without password → SQLi confirmed!

  Burp's active scanner will also auto-flag this as
  "SQL Injection" with a HIGH severity rating.
```

SLIP 17

Q1 — Product Cart (PHP)

REPEAT: Identical to Slip 14 Q1. Refer to Slip 14 Q1 for full solution.

Q2 — Course Registrations.xml (5 records)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<CourseRegistrations>
  <Registration>
    <RegistrationID>REG001</RegistrationID>
    <StudentName>Rushikesh Bhosale</StudentName>
    <CourseName>Ethical Hacking</CourseName>
    <Instructor>Prof. Sharma</Instructor>
    <DateOfRegistration>2025-01-15</DateOfRegistration>
  </Registration>
  <Registration>
    <RegistrationID>REG002</RegistrationID>
    <StudentName>Priya Patil</StudentName>
    <CourseName>Web Development</CourseName>
    <Instructor>Prof. Kulkarni</Instructor>
    <DateOfRegistration>2025-01-16</DateOfRegistration>
  </Registration>
  <Registration>
    <RegistrationID>REG003</RegistrationID>
    <StudentName>Rohit Desai</StudentName>
    <CourseName>Cloud Computing</CourseName>
    <Instructor>Prof. Joshi</Instructor>
    <DateOfRegistration>2025-01-17</DateOfRegistration>
  </Registration>
  <Registration>
    <RegistrationID>REG004</RegistrationID>
    <StudentName>Sneha More</StudentName>
    <CourseName>Digital Forensics</CourseName>
    <Instructor>Prof. Nair</Instructor>
    <DateOfRegistration>2025-01-18</DateOfRegistration>
  </Registration>
  <Registration>
    <RegistrationID>REG005</RegistrationID>
    <StudentName>Aman Verma</StudentName>
    <CourseName>Network Security</CourseName>
    <Instructor>Prof. Mehta</Instructor>
    <DateOfRegistration>2025-01-19</DateOfRegistration>
  </Registration>
</CourseRegistrations>
```

SLIP 18

Q1 — breakfast.xml + add Juice category

breakfast.xml (initial):

```
<?xml version="1.0" encoding="UTF-8"?>
<Breakfast>
  <Item category="Fast Food">
    <Name>French Fries</Name>
    <Price>45</Price>
    <Description>Young youths are very much interested to eat it</Description>
    <Calories>650</Calories>
  </Item>
</Breakfast>
```

Adding Juice category (append this inside <Breakfast>):

```
  <Item category="Juice">
    <Name>Orange Juice</Name>
    <Price>60</Price>
    <Description>Fresh squeezed orange juice, rich in Vitamin C</Description>
    <Calories>110</Calories>
  </Item>
  <Item category="Juice">
    <Name>Mango Shake</Name>
    <Price>80</Price>
    <Description>Thick and creamy mango shake</Description>
    <Calories>200</Calories>
  </Item>
```

Q2 — SQL Injection via UPDATE Query (Unauthorized Profile Modification)

```
ATTACK: SQL Injection on UPDATE Statement
==========================================

Vulnerable PHP code:
  $query = "UPDATE users SET email='$email' WHERE username='$username'";

Normal input:
  email    = newemail@gmail.com
  username = alice
  → UPDATE users SET email='newemail@gmail.com' WHERE username='alice'

SQL Injection Payload:
  email    = hacked@attacker.com', admin='1
  username = alice

  → UPDATE users SET email='hacked@attacker.com', admin='1'
    WHERE username='alice'
  → This modifies MULTIPLE columns including the admin flag!

Another Payload (affect all rows):
  username = ' OR '1'='1
  → UPDATE users SET email='...' WHERE username='' OR '1'='1'
  → Modifies ALL user records in the table!

Prevention:
```

```
Use prepared statements:
$stmt = $pdo->prepare("UPDATE users SET email=? WHERE username=?");
$stmt->execute([$email, $username]);
```

REPEAT: Slip 25 Q1 is the same UPDATE injection. Refer here.

SLIP 19

Q1 — Book Details Form + PHP MySQL Storage

book_form.html:

```
<form action="save_book.php" method="POST">
  Book No   : <input type="text"   name="bno"><br>
  Book Name : <input type="text"   name="bname"><br>
  Price     : <input type="number" name="price" step="0.01"><br>
  <button type="submit">Save Book</button>
</form>
```

save_book.php:

```
<?php
$conn = new mysqli("localhost", "root", "", "library_db");
if ($conn->connect_error) { die("Connection failed: " . $conn->connect_error); }

// Create table if not exists
$conn->query("CREATE TABLE IF NOT EXISTS book (
  bno    INT PRIMARY KEY,
  bname  VARCHAR(100),
  price  DECIMAL(8,2)
)");

$bno   = $_POST['bno'];
$bname = $_POST['bname'];
$price = $_POST['price'];

$stmt = $conn->prepare("INSERT INTO book (bno, bname, price) VALUES (?,?,?)");
$stmt->bind_param("isd", $bno, $bname, $price);

if ($stmt->execute()) {
  echo "Book saved successfully!<br>";
  echo "Book No: $bno | Name: $bname | Price: Rs.$price";
} else {
  echo "Error: " . $stmt->error;
}
$stmt->close(); $conn->close();
?>
```

REPEAT: Q2 of this slip (Dairy Product Service) is same as Slip 3 Q2. Refer to Slip 3 Q2.

SLIP 20

Q1 — Library Book Web Service (PHP REST API)

File: library_service.php

```php
<?php
header('Content-Type: application/json');
$file = 'books.json';
$books = file_exists($file) ? json_decode(file_get_contents($file), true) : [];

$action = $_GET['action'] ?? 'list';
$id     = $_GET['id']     ?? null;

// Get POST data
$title  = $_POST['title']  ?? '';
$author = $_POST['author'] ?? '';
$genre  = $_POST['genre']  ?? '';
$price  = $_POST['price']  ?? '';

switch ($action) {
  case 'add':
    $books[] = ['id'=>uniqid(), 'title'=>$title, 'author'=>$author,
                'genre'=>$genre, 'price'=>$price];
    file_put_contents($file, json_encode($books, JSON_PRETTY_PRINT));
    echo json_encode(['status'=>'Book added']);
    break;

  case 'delete':
    $books = array_values(array_filter($books, fn($b) => $b['id'] !== $id));
    file_put_contents($file, json_encode($books, JSON_PRETTY_PRINT));
    echo json_encode(['status'=>'Book deleted']);
    break;

  case 'update':
    foreach ($books as &$b) {
      if ($b['id'] === $id) {
        $b['title']=$title; $b['author']=$author;
        $b['genre']=$genre; $b['price']=$price;
        break;
      }
    }
    file_put_contents($file, json_encode($books, JSON_PRETTY_PRINT));
    echo json_encode(['status'=>'Book updated']);
    break;

  case 'get':
    if ($id) {
      $result = array_values(array_filter($books, fn($b) => $b['id'] === $id));
      echo json_encode($result[0] ?? ['error'=>'Not found']);
    } else {
      echo json_encode($books);  // Get all books
    }
    break;

  default:
    echo json_encode($books);
}
?>
```

 REPEAT: Slip 26 Q2 is identical. Refer here.

Q2 — Secure User Registration

 Same as Slip 5 Q1 (Registration with encrypted password). Refer to Slip 5 Q1 for complete solution.

SLIP 21

Q1 — Factorial using PHP Function

```
<!-- factorial.html -->
<form action="factorial.php" method="POST">
  Enter Number: <input type="number" name="num" min="0">
  <button type="submit">Calculate Factorial</button>
</form>
```

```
<?php
// factorial.php
function factorial($n) {
  if ($n == 0 || $n == 1) return 1;
  return $n * factorial($n - 1);
}

$num = $_POST['num'];
if ($num < 0) {
  echo "Factorial is not defined for negative numbers.";
} else {
  $result = factorial($num);
  echo "Factorial of $num = $result";
}
?>
```

Q2 — OWASP ZAP Scanner: Short Note with Example

```
OWASP ZAP (Zed Attack Proxy) — Short Note
==========================================

Definition:
  OWASP ZAP is a free, open-source web application security scanner
  maintained by OWASP (Open Web Application Security Project).
  It is widely used for finding vulnerabilities in web apps.

Key Features:
  - Passive Scanner  : Detects issues without sending malicious requests
  - Active Scanner   : Sends attack payloads to find vulnerabilities
  - Spider/Crawler   : Automatically discovers all pages of a web app
  - Fuzzer           : Sends random/invalid inputs to find crashes
  - Proxy            : Intercept and modify HTTP traffic

Common Vulnerabilities ZAP Detects:
  - SQL Injection
  - Cross-Site Scripting (XSS)
  - Sensitive Data Exposure
  - Broken Access Control
  - CSRF (Cross-Site Request Forgery)

Example:
  Step 1: Launch ZAP → Set browser proxy to localhost:8080
  Step 2: Enter target URL (e.g., http://localhost/login.php)
  Step 3: ZAP → Automated Scan → Click "Attack"
  Step 4: ZAP crawls all pages and sends test payloads
  Step 5: Alerts panel shows:
          [HIGH]   SQL Injection in parameter 'username'
          [MEDIUM] X-Frame-Options header not set
```

```
          [LOW]    Cookie without HttpOnly flag
  Step 6: Click each alert for details and remediation advice

ZAP vs Burp Suite:
  ZAP = Free + Open Source (good for students/beginners)
  Burp = Commercial (more powerful scanners in Pro version)
```

SLIP 22

Q1 — OWASP Top 10 API Security Threats

```
OWASP API Security Top 10 (2023)
================================

1. API1 - Broken Object Level Authorization (BOLA/IDOR)
   Attacker accesses other users' data by manipulating object IDs.
   Example: GET /api/users/1001/data → change 1001 to 1002 to see other user's data.

2. API2 - Broken Authentication
   Weak tokens, no expiry, or missing auth on API endpoints.
   Example: JWT token with no expiry allows indefinite access.

3. API3 - Broken Object Property Level Authorization
   API returns more fields than needed; attacker exploits extra data.
   Example: API returns password_hash field in user profile response.

4. API4 - Unrestricted Resource Consumption
   No rate limiting → server overload via massive API calls (DoS).
   Example: Sending 10,000 requests/sec to OTP endpoint.

5. API5 - Broken Function Level Authorization
   Non-admin users call admin-only API functions.
   Example: Regular user calls DELETE /api/admin/users/5.

6. API6 - Unrestricted Access to Sensitive Business Flows
   Automated abuse of legitimate business flows.
   Example: Bot buying all concert tickets before humans can.

7. API7 - Server-Side Request Forgery (SSRF)
   API fetches attacker-controlled URLs → internal network exposure.
   Example: ?url=http://169.254.169.254/latest/meta-data/ (AWS metadata).

8. API8 - Security Misconfiguration
   Debug endpoints exposed, default credentials, unnecessary HTTP methods enabled.
   Example: /api/debug returns full stack trace with DB credentials.

9. API9 - Improper Inventory Management
   Old/undocumented API versions remain active and unpatched.
   Example: /api/v1/users still works and has SQLi vulnerability.

10. API10 - Unsafe Consumption of APIs
    Trusting third-party APIs without validation.
    Example: App blindly uses data from external API → XSS injection via API response.
```

REPEAT: Q2 is Password Validator — same as Slip 4 Q2. Refer to Slip 4 Q2.

SLIP 23

Q1 — Employee Profile Webpage with Contact HR & Add Employee Form

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Employee Profile</title>
<style>
  body { font-family: Arial; margin: 0; }
  .container { display: flex; min-height: 100vh; }
  .nav { width: 200px; background: #1a3c5e; padding: 20px; }
  .nav a { display: block; color: white; padding: 10px; margin: 5px 0;
           text-decoration: none; border-radius: 4px; }
  .nav a:hover { background: #2e75b6; }
  .content { flex: 1; padding: 30px; background: #f0f4f8; }
  img.photo { width: 100px; height: 100px; border-radius: 50%;
              object-fit: cover; border: 3px solid #2e75b6; }
  table { border-collapse: collapse; margin: 10px 0; }
  td { padding: 8px 15px; border-bottom: 1px solid #ccc; }
  input, select { padding: 6px; margin: 4px; border: 1px solid #ccc;
                  border-radius: 4px; width: 200px; }
  button { padding: 8px 20px; background: #2e75b6; color: white;
           border: none; border-radius: 4px; cursor: pointer; }
  #hr-info, #add-form { display: none; margin-top: 20px;
                        background: white; padding: 15px; border-radius: 6px; }
</style>
</head>
<body>
<div class="container">
  <!-- Column 1: Navigation -->
  <div class="nav">
    <h3 style="color:white;">Menu</h3>
    <a href="#" onclick="showSection('profile')">□ Home</a>
    <a href="#" onclick="showSection('about')">□ About Us</a>
    <a href="#" onclick="showSection('hr')">□ Contact HR</a>
    <a href="#" onclick="showSection('addform')">➕ Add Employee</a>
  </div>

  <!-- Column 2: Employee Details -->
  <div class="content">
    <div id="profile">
      <h2>Employee Profile</h2>
      <img class="photo" src="employee.jpg" alt="Employee Photo">
      <table>
        <tr><td><b>Employee ID</b></td><td>EMP-1001</td></tr>
        <tr><td><b>Name</b></td><td>Rushikesh Bhosale</td></tr>
        <tr><td><b>Designation</b></td><td>SOC Analyst</td></tr>
        <tr><td><b>Department</b></td><td>Cybersecurity</td></tr>
        <tr><td><b>Email</b></td><td>rushikesh@company.com</td></tr>
      </table>
    </div>

    <!-- HR Contact Info -->
    <div id="hr-info">
      <h3>HR Department</h3>
      <p>Department : Human Resources</p>
      <p>Email      : hr@company.com</p>
```

```
      <p>Phone      : +91 9876500000</p>
    </div>

    <!-- Add Employee Form -->
    <div id="add-form">
      <h3>Add New Employee</h3>
      <input type="text"   placeholder="Name"><br>
      <input type="text"   placeholder="Employee ID"><br>
      <input type="text"   placeholder="Designation"><br>
      <input type="text"   placeholder="Department"><br>
      <input type="email"  placeholder="Email"><br>
      <input type="tel"    placeholder="Phone Number"><br><br>
      <button>Add Employee</button>
    </div>
  </div>
</div>

<script>
  function showSection(section) {
    document.getElementById('profile').style.display  = 'none';
    document.getElementById('hr-info').style.display  = 'none';
    document.getElementById('add-form').style.display = 'none';
    if (section === 'profile') document.getElementById('profile').style.display  =
'block';
    if (section === 'hr')      document.getElementById('hr-info').style.display  =
'block';
    if (section === 'addform') document.getElementById('add-form').style.display =
'block';
  }
</script>
</body>
</html>
```

Q2 — Feedback Form with Star Rating

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Feedback Form</title>
<style>
  body { font-family: Arial; padding: 20px; background: #f5f5f5; }
  .box { background: white; padding: 25px; border-radius: 8px; max-width: 500px; margin:
auto; }
  textarea { width: 100%; height: 80px; }
  input, textarea, select { padding: 7px; margin: 5px 0; border: 1px solid #ccc;
                            border-radius: 4px; width: 100%; box-sizing: border-box; }
  button { padding: 10px 25px; background: #2e75b6; color: white;
           border: none; border-radius: 4px; cursor: pointer; }
  #result { margin-top: 20px; padding: 15px; background: #e8f4f8; border-radius: 6px;
display: none; }
  .stars { color: gold; font-size: 24px; }
</style>
</head>
<body>
<div class="box">
  <h2>Feedback Form</h2>
  Name    : <input type="text" id="name" placeholder="Your Name"><br>
  Email   : <input type="email" id="email" placeholder="Your Email"><br>
  Feedback: <textarea id="feedback" placeholder="Your feedback..."></textarea>
  Rating (1-5):
```

```
  <select id="rating">
    <option value="1">1 - Poor</option>
    <option value="2">2 - Fair</option>
    <option value="3">3 - Good</option>
    <option value="4" selected>4 - Very Good</option>
    <option value="5">5 - Excellent</option>
  </select><br><br>
  <button onclick="submitFeedback()">Submit Feedback</button>

  <div id="result">
    <h3>Thank You!</h3>
    <p><b>Name:</b>     <span id="r-name"></span></p>
    <p><b>Email:</b>    <span id="r-email"></span></p>
    <p><b>Feedback:</b> <span id="r-feedback"></span></p>
    <p><b>Rating:</b>   <span class="stars" id="r-stars"></span></p>
  </div>
</div>

<script>
  function submitFeedback() {
    const name     = document.getElementById('name').value;
    const email    = document.getElementById('email').value;
    const feedback = document.getElementById('feedback').value;
    const rating   = parseInt(document.getElementById('rating').value);

    document.getElementById('r-name').textContent     = name;
    document.getElementById('r-email').textContent    = email;
    document.getElementById('r-feedback').textContent = feedback;
    document.getElementById('r-stars').textContent    = '★'.repeat(rating) + '☆'.repeat(5
- rating);
    document.getElementById('result').style.display   = 'block';
  }
</script>
</body>
</html>
```

SLIP 24

Q1 — Timetable.xml

REPEAT: Same as Slip 15 Q2. Refer to Slip 15 Q2 for the complete XML.

Q2 — Currency Exchange API (PHP)

```php
<?php
// Using exchangerate-api.com (free tier available)
$apiKey  = "YOUR_API_KEY";
$from    = $_POST['from']   ?? 'USD';
$to      = $_POST['to']     ?? 'INR';
$amount  = $_POST['amount'] ?? 1;

$url  = "https://v6.exchangerate-api.com/v6/{$apiKey}/pair/{$from}/{$to}/{$amount}";
$res  = @file_get_contents($url);

if ($res === false) {
  echo "Error: Could not reach API. Check API key or currency codes.";
} else {
  $data = json_decode($res, true);

  if ($data['result'] === 'success') {
    $rate      = $data['conversion_rate'];
    $converted = $data['conversion_result'];
    echo "Exchange Rate: 1 {$from} = {$rate} {$to}<br>";
    echo "Converted Amount: {$amount} {$from} = {$converted} {$to}";
  } else {
    echo "Error: " . $data['error-type'];   // Invalid currency code etc.
  }
}
?>

<!-- HTML Form -->
<form method="POST">
  From Currency : <input type="text" name="from" value="USD" maxlength="3"><br>
  To Currency   : <input type="text" name="to"   value="INR" maxlength="3"><br>
  Amount        : <input type="number" name="amount" value="1"><br>
  <button type="submit">Convert</button>
</form>
```

Get a free API key from exchangerate-api.com. Handles invalid codes via error-type field.

SLIP 25 & SLIP 26 — Repeat Slips

SLIP 25

Q1 — SQL Injection UPDATE statement

REPEAT: Identical to Slip 18 Q2. Refer to Slip 18 Q2 for complete explanation and payloads.

Q2 — cricket.xml with Australia team

REPEAT: Identical to Slip 3 Q1. Refer to Slip 3 Q1 for the complete XML.

SLIP 26

Q1 — Student.xml with 5 records

REPEAT: Identical to Slip 10 Q1. Refer to Slip 10 Q1 for the complete XML.

Q2 — Library Book Web Service

REPEAT: Identical to Slip 20 Q1. Refer to Slip 20 Q1 for the complete PHP code.

Summary of All Repeated Questions

```
Repeated Questions — Quick Reference
====================================
Topic                          | Solved In  | Repeated In
-------------------------------|------------|-------------------
Student Profile Webpage        | Slip 1 Q1  | Slip 5 Q2, Slip 23
Weather API (PHP)              | Slip 4 Q1  | Slip 7 Q2
Password Validator (HTML/JS)   | Slip 4 Q2  | Slip 8 Q2, Slip 22 Q2
SQL Injection on Login         | Slip 2 Q2  | Slip 16 Q1
Dairy Product Web Service      | Slip 3 Q2  | Slip 19 Q2
Product Cart (PHP)             | Slip 14 Q1 | Slip 17 Q1
Timetable.xml                  | Slip 15 Q2 | Slip 24 Q1
SQL Injection UPDATE           | Slip 18 Q2 | Slip 25 Q1
cricket.xml (Australia)        | Slip 3 Q1  | Slip 25 Q2
Student.xml (5 records)        | Slip 10 Q1 | Slip 26 Q1
Library Book Web Service       | Slip 20 Q1 | Slip 26 Q2
Secure User Registration       | Slip 5 Q1  | Slip 20 Q2
```

All viva questions (Q3, 5 marks each) are oral — no written solution required. Study topics from your slips!